Modul Praktikum 3

CLASS AND OBJECT

D3 Teknik Informatika

POLITEKNIK NEGERI BANDUNG

Daftar Isi

Daftar Isi .....2 1. Modul Overview .....4 1.1 Tujuan Pembelajaran.....4 1.2 Target Milestones.....4 1.3 Alur Belajar Praktikum .....4 1.4 Prasyarat.....5 2. Kasus Praktikum: Kantin Kampus Mini .....5 2.1 Batas Kasus.....5 2.2 Aturan Kasus .....5 2.3 Model Awal.....5 3. Peta Konsep Implementasi Kode Java .....6 3.1 Class, State, dan Behavior.....6 3.2 Object / Instance .....6 3.3 Encapsulation: Menjaga State.....6 3.4 Constructor: Object Lahir dengan State Awal .....7 3.5 Getter: Membaca Bukan Berarti Bebas Mengubah.....7 3.6 Static: Milik Class, Bukan Milik Setiap Object.....7 3.7 Relationship antar-Class melalui Object Reference.....8 3.8 Message / Method Call dalam Kolaborasi Object.....8 3.9 Package dan

Import .....9 4. Praktikum
Terbimbing: Membangun Program Bertahap.....9 Tahap 1 -
Bangun Class Mahasiswa .....9 Tahap 2 -
Bangun MenuItem dan Uji Encapsulation.....10 Tahap 3 -
Bangun Pesanan dan Relationship .....10 Tahap 4 -
Buat dan Jalankan Object yang Berkolaborasi .....11 Tahap 5 -
Uji Static secara Sengaja .....11 Tahap 6 -
Susun Package .....12 5. Eksperimen
Teknis .....12 Eksperimen 1 -
private vs public .....12 Eksperimen 2 -
Constructor vs Object Setengah Jadi.....12 Eksperimen 3 -
static vs instance.....12 Eksperimen 4 -
Relationship: Reference vs Duplikasi Data.....13 Eksperimen 5 -
Package dan Compiler .....13 6. Tugas Praktikum
Terintegrasi.....13 Task 1 - Lengkapi
Model Teknis .....13 Task 2 - Buktikan
Class dan Object .....13 Task 3 - Buktikan
Encapsulation dan Constructor 13

Task 4 - Buktikan Static 14
Task 5 - Buktikan Relationship 14
Task 6 - Package Final.....14
6.1 Skenario Uji Wajib 14
6.2 Bukti Wajib 14
7. Independent Challenge.....15 7.1
Challenge A - Relationship uses-a dengan Kasir 15 7.2
Challenge B - Pertanyaan Menuju Inheritance 15
8. Checkpoint, Penilaian, dan Refleksi.....15
8.1 Definition of Done 15 8.2
Self Assessment 15 8.3
Lesson Learnt .....16 9.
Referensi 16

1. Modul Overview

Pada dua pertemuan awal mahasiswa telah mengenal class, object, state/attribute, behavior/method, message, dan encapsulation. Modul ini mengubah konsep tersebut menjadi pengalaman teknis: mahasiswa menulis class, membuat object, melihat compiler menolak akses yang tidak sah, menggunakan constructor, membedakan state per-instance dengan state shared melalui static, menghubungkan object melalui reference, lalu mengorganisasi source code menggunakan package.

Tujuan utama praktikum bukan menghafal bentuk syntax. Setiap potongan kode harus selalu dijawab dengan tiga pertanyaan: apa konsep yang sedang direpresentasikan, mengapa struktur kode itu diperlukan, dan apa yang terjadi jika struktur tersebut diubah.

1.1 Tujuan Pembelajaran

1) Menerjemahkan model sederhana menjadi class Java yang memiliki state dan behavior yang jelas. 2) Membuat beberapa object/instance dari class yang sama dan membuktikan bahwa setiap object dapat memiliki state berbeda.
3) Menerapkan encapsulation menggunakan access modifier dan behavior yang bermakna, bukan sekadar membuat semua field private lalu menambahkan setter.
4) Menggunakan constructor untuk memastikan object menerima state awal yang dibutuhkan saat dibuat. 5)

Membedakan instance field/method dan static field/method berdasarkan siapa yang memiliki data atau behavior tersebut.
6) Merepresentasikan relationship antar-class melalui object reference dan method call.
7) Mengorganisasi class ke dalam package serta menggunakan import secara benar.
8) Menjelaskan hubungan antara konsep OOP dan baris kode yang ditulis dengan bahasa sendiri.

1.2 Target Milestones

Milestone	Bukti Teknis yang Harus Dapat Ditunjukkan
Class Ready	Class memiliki field dan method yang dapat dijelaskan perannya.
Object Ready	Minimal dua instance dari satu class memiliki state yang berbeda.
Encapsulation Ready	Akses langsung ke field private gagal; perubahan state dilakukan melalui behavior yang disediakan object.
Construction Ready	Object dibuat melalui constructor dengan data awal yang diperlukan.
Static Ready	Mahasiswa dapat membuktikan perbedaan nilai instance dan nilai shared/static melalui output program.
Relationship Ready	Satu class menyimpan reference ke object class lain dan berkolaborasi melalui method call.
Package Ready	Class berada pada package yang benar dan program berjalan menggunakan import yang sesuai.

1.3 Alur Belajar Praktikum

Tahap	Aktivitas
Concept	Pahami makna konsep pada kasus.
Map	Tentukan bentuk representasinya: class, field, method, constructor, reference, atau static.
Code	Tulis perubahan kode kecil.
Run	Compile dan jalankan.
Observe	Amati output atau error compiler.
Explain	Jelaskan mengapa hasil tersebut terjadi.
Modify	Ubah satu bagian dan prediksi dampaknya sebelum dijalankan kembali.

Aturan penggunaan AI

AI boleh digunakan setelah mahasiswa memiliki prediksi dan percobaan sendiri. Mahasiswa harus mampu menjelaskan setiap baris utama dan melakukan perubahan kecil pada kode tanpa AI.

1.4 Prasyarat

- JDK dan IDE/editor Java sudah dapat digunakan.
- Mahasiswa sudah pernah menulis class sederhana dan method dasar.
- Tidak menggunakan database, GUI, framework, atau library eksternal.
- Satu folder/repository digunakan selama praktikum.

2. Kasus Praktikum: Kantin Kampus Mini

Sebuah kantin kampus membutuhkan program sederhana untuk mencatat pesanan. Seorang mahasiswa memilih satu menu. Setiap menu mempunyai kode, nama, harga, dan kondisi tersedia/habis. Sebuah pesanan menyimpan siapa

pemesannya, menu yang dipilih, dan jumlah yang dipesan. Total dihitung dari harga menu × jumlah. Setiap pesanan memiliki nomor urut yang dibuat otomatis.

2.1 Batas Kasus

1) Satu pesanan hanya memuat satu jenis menu.
2) Tidak ada database, login, pembayaran, diskon, stok numerik, GUI, atau banyak tenant.
3) Status menu hanya tersedia/habis.
4) Inheritance belum diimplementasikan pada modul ini; is-a hanya dikenali sebagai jenis relationship untuk materi berikutnya.

2.2 Aturan Kasus

ID	Aturan
BR-01	Mahasiswa memiliki NIM dan nama.
BR-02	Menu memiliki kode, nama, harga, dan status tersedia/habis.
BR-03	Field domain tidak diakses langsung dari luar object.
BR-04	Pesanan menyimpan reference ke object Mahasiswa dan MenuItem, bukan menyalin seluruh datanya.
BR-05	Jumlah pesanan minimal 1.
BR-06	Menu yang habis tidak dapat diproses sebagai pesanan yang valid.
BR-07	Nomor pesanan meningkat untuk setiap object Pesanan yang dibuat.

2.3 Model Awal

Sebelum melihat kode

Gambarkan maksimal 3 class yang menurut Anda diperlukan. Untuk setiap class, tuliskan state dan behavior. Setelah itu, bandingkan model Anda dengan implementasi bertahap pada bagian berikutnya.

Class	State yang Mungkin Dibutuhkan	Behavior yang Mungkin Dibutuhkan
Mahasiswa	nim, nama	menyediakan identitas untuk kebutuhan program
MenuItem	kode, nama, harga, tersedia	memberi informasi harga/ketersediaan; mengubah ketersediaan melalui behavior
Pesanan	nomor, pemesan, menu, jumlah	memeriksa dapat diproses, menghitung total, menampilkan ringkasan

Tabel di atas adalah checkpoint model, bukan satu-satunya jawaban yang sah. Fokusnya adalah apakah setiap state dan behavior memiliki alasan yang dapat dijelaskan.

3. Peta Konsep Implementasi Kode

Konsep	Makna pada Kasus	Representasi Java

Class	Definisi jenis object	class Mahasiswa { ... }
Object / Instance	Satu wujud konkret dari class	new Mahasiswa("241001", "Asep")
State / Attribute	Data yang dimiliki object	private String nim;
Behavior / Method	Apa yang dapat dilakukan object	public boolean isTersedia()
Encapsulation	Object mengendalikan state-nya	private field + method yang relevan
Constructor	Cara object menerima state awal	public Mahasiswa(String nim, String nama)
Static	Data/behavior milik class, shared antar-instance	private static int counter;
Relationship	Object mengenal/menggunakan object lain	private Mahasiswa pemesan;
Package	Pengelompokan class dalam namespace	package id.ac.polban.pbo.kantin.model;

3.1 Class, State, dan Behavior

Mulai dari class paling sederhana. Mahasiswa memiliki dua state: NIM dan nama. Pada tahap ini class belum harus memiliki banyak method. Tujuan pertama adalah melihat hubungan antara model konseptual dan struktur source code.

Bentuk awal class Mahasiswa

```
public class Mahasiswa {
 private String nim;
 private String nama;
}
```

Baca kodenya

class mendefinisikan bentuk object. nim dan nama adalah state yang nantinya dimiliki setiap instance. Kata private mulai membatasi akses state dari luar object.

3.2 Object / Instance

Class belum menjadi data mahasiswa tertentu. Object dibuat dengan operator new. Dua object dari class yang sama dapat membawa state yang berbeda.

Dua instance dari class yang sama

```
Mahasiswa m1 = new Mahasiswa("241001", "Asep");
Mahasiswa m2 = new Mahasiswa("241002", "Siti");
```

Pertanyaan observasi

Apa yang sama antara m1 dan m2? Apa yang berbeda? Bagian mana yang merupakan class, object reference, dan object yang dibuat oleh new?

3.3 Encapsulation: Menjaga State

Encapsulation tidak berhenti pada kata private. private hanya mencegah akses langsung. Desain yang lebih penting adalah menentukan behavior apa yang memang boleh mengubah state.

Contoh state yang dijaga melalui behavior

```
public class MenuItem {
 private String kode;
 private String nama;
 private int harga;
 private boolean tersedia;
```



```
 public boolean isTersedia() {
 return tersedia;
 }

 public void tandaiHabis() {
 tersedia = false;
 }
}
```

Dengan struktur tersebut, bagian lain program tidak perlu menulis menu.tersedia = false. Bagian lain cukup meminta object melakukan behavior tandaiHabis().

Eksperimen akses state

```
// Ini harus gagal dikompilasi dari class lain:
menu.tersedia = false;

// Ini yang diperbolehkan oleh desain object:
menu.tandaiHabis();
```

3.4 Constructor: Object Lahir dengan State Awal

Tanpa constructor yang sesuai, object berisiko dibuat tanpa data penting. Constructor digunakan untuk menerima nilai awal ketika new dipanggil.

Constructor Mahasiswa

```
public Mahasiswa(String nim, String nama) {
 this.nim = nim;
 this.nama = nama;
}
```

this.nim menunjuk field milik object saat ini, sedangkan nim di sisi kanan adalah parameter constructor. Setelah constructor ini didefinisikan, pemanggilan new Mahasiswa() tanpa argument tidak lagi cocok dengan constructor yang tersedia.

3.5 Getter: Membaca Bukan Berarti Bebas Mengubah

Accessor untuk kebutuhan baca

```
public String getNim() {
 return nim;
}

public String getNama() {
 return nama;
}
```

Catatan desain

Tidak semua field otomatis membutuhkan setter. Jika perubahan state seharusnya mengikuti aturan tertentu, gunakan behavior yang bermakna daripada setter umum.

3.6 Static: Milik Class, Bukan Milik Setiap Object

Nomor pesanan perlu terus meningkat ketika object Pesanan baru dibuat. Counter bukan state unik milik satu pesanan; counter dibagi oleh seluruh instance Pesanan. Karena itu counter cocok menjadi static.

Counter shared menggunakan static

```
public class Pesanan {
 private static int nextNumber = 1;
```



```
 private int nomor;

 public Pesanan() {
 nomor = nextNumber;
 nextNumber++;
 }
}
```

Setiap object memiliki field nomor sendiri, tetapi seluruh object membaca dan mengubah nextNumber yang sama. Jika nextNumber bukan static, setiap object akan memiliki counter sendiri dan pola nomor tidak akan meningkat sebagaimana yang diharapkan.

Prediksi sebelum Run

```
Pesanan p1 = new Pesanan();
Pesanan p2 = new Pesanan();
Pesanan p3 = new Pesanan();

// Prediksi: nomor p1, p2, p3?
```

3.7 Relationship antar-Class melalui Object Reference

Dalam OOP, object tidak harus menyalin semua data object lain. Pesanan dapat menyimpan reference ke object Mahasiswa dan MenuItem yang memang sudah ada.

Relationship has-a pada level kode

```
public class Pesanan {
 private Mahasiswa pemesan;
 private MenuItem menu;
 private int jumlah;

 public Pesanan(Mahasiswa pemesan, MenuItem menu, int jumlah) {
 this.pemesan = pemesan;
 this.menu = menu;
 this.jumlah = jumlah;
 }
}
```

Field pemesan dan menu bukan String. Keduanya bertipe class lain. Secara konseptual Pesanan memiliki/mengenal object Mahasiswa dan MenuItem. Ini adalah cara relationship diwujudkan melalui reference.

Relationship	Makna Singkat	Contoh pada Modul
has-a	Object menyimpan reference sebagai bagian dari state-nya.	Pesanan menyimpan pemesan dan menu.
uses-a	Object hanya menggunakan object lain untuk suatu operasi, misalnya sebagai parameter method, tanpa harus menyimpannya.	Challenge: Kasir.proses(Pesanan p).
is-a	Satu class merupakan jenis yang lebih khusus dari class lain.	Belum diimplementasikan; dibahas saat inheritance.

3.8 Message / Method Call dalam Kolaborasi Object

Pesanan meminta informasi dari MenuItem

```
public boolean dapatDiproses() {
 return jumlah > 0 && menu.isTersedia();
}

public int hitungTotal() {
 return menu.getHarga() * jumlah;
}
```


Pada menu.isTersedia() dan menu.getHarga(), object Pesanan mengirim message melalui method call kepada object MenuItem. Pesanan tidak perlu mengetahui bagaimana MenuItem menyimpan state-nya.

3.9 Package dan Import

Saat class mulai bertambah, source code dikelompokkan menggunakan package. Deklarasi package harus sesuai dengan organisasi source code pada proyek/IDE.

Struktur package target

```
src/
└── id/ac/polban/pbo/kantin/
    ├── model/
    │   ├── Mahasiswa.java
    │   ├── MenuItem.java
    │   └── Pesanan.java
    └── app/
        └── Main.java
```

Deklarasi package pada class model

```
// Mahasiswa.java
package id.ac.polban.pbo.kantin.model;

public class Mahasiswa {
 // ...
}
```

Import dari package app

```
package id.ac.polban.pbo.kantin.app;

import id.ac.polban.pbo.kantin.model.Mahasiswa;
import id.ac.polban.pbo.kantin.model.MenuItem;
import id.ac.polban.pbo.kantin.model.Pesanan;

public class Main {
 public static void main(String[] args) {
 // membuat dan menghubungkan object
 }
}
```

4. Praktikum Terbimbing: Membangun Program Bertahap

Kerjakan tahap berikut secara berurutan. Jangan menyalin seluruh program sekaligus. Setelah setiap tahap, compile/run dan jawab pertanyaan observasi sebelum melanjutkan.

Tahap 1 - Bangun Class Mahasiswa

1. Buat file Mahasiswa.java.
2. Tambahkan field private nim dan nama.
3. Buat constructor Mahasiswa(String nim, String nama).
4. Tambahkan getNim() dan getNama().
5. Di Main, buat dua object Mahasiswa dengan data berbeda.
6. Cetak data keduanya melalui getter.

Starter Main - Tahap 1

```
Mahasiswa m1 = new Mahasiswa("241001", "Asep");
Mahasiswa m2 = new Mahasiswa("241002", "Siti");

System.out.println(m1.getNim() + " - " + m1.getNama());
System.out.println(m2.getNim() + " - " + m2.getNama());
```

Prediksi / Pertanyaan	Jawaban Mahasiswa
Apakah m1 dan m2 menggunakan class yang sama?	
Mengapa nilai state-nya bisa berbeda?	
Apa yang terjadi jika menulis System.out.println(m1.nama); dari Main?	

Tahap 2 - Bangun MenuItem dan Uji Encapsulation

1. Buat MenuItem dengan field private: kode, nama, harga, tersedia.
2. Buat constructor yang menerima kode, nama, harga; state tersedia diinisialisasi true.
3. Tambahkan getter yang diperlukan.

4. Tambahkan behavior isTersedia() dan tandaiHabis().
5. Buat dua object MenuItem. Tandai salah satunya habis.
6. Buktikan bahwa state kedua object tidak saling memengaruhi.

Constructor MenuItem

```
public MenuItem(String kode, String nama, int harga) {
this.kode = kode;
this.nama = nama;
this.harga = harga;
this.tersedia = true;
}
```

Uji dua object dengan state berbeda

```
MenuItem nasi = new MenuItem("M01", "Nasi Goreng", 18000);
MenuItem kopi = new MenuItem("M02", "Kopi Susu", 12000);

kopi.tandaiHabis();

System.out.println(nasi.getNama() + " tersedia: " + nasi.isTersedia());
System.out.println(kopi.getNama() + " tersedia: " + kopi.isTersedia());
```

Eksperimen wajib

Coba akses kopi.tersedia langsung dari Main. Catat pesan compiler. Jangan hanya menulis “error”; jelaskan konsep apa yang sedang ditegakkan oleh compiler.

Tahap 3 - Bangun Pesanan dan Relationship

Sekarang object mulai berkolaborasi. Pesanan tidak menyimpan nim, nama mahasiswa, nama menu, dan harga sebagai copy terpisah. Simpan reference ke object yang relevan.

Starter Pesanan - lengkapi getter/ringkasan sesuai kebutuhan

```
public class Pesanan {
private static int nextNumber = 1;

private int nomor;
private Mahasiswa pemesan;
private MenuItem menu;
```



```
private int jumlah;

public Pesanan(Mahasiswa pemesan, MenuItem menu, int jumlah) {
this.nomor = nextNumber++;
this.pemesan = pemesan;
this.menu = menu;
this.jumlah = jumlah;
}

public boolean dapatDiproses() {
return jumlah > 0 && menu.isTersedia();
}

public int hitungTotal() {
return menu.getHarga() * jumlah;
}
}
```

Baris Kode	Makna Konseptual
private Mahasiswa pemesan;	Pesanan memiliki reference ke object Mahasiswa.
private MenuItem menu;	Pesanan memiliki reference ke object MenuItem.
this.pemesan = pemesan;	Reference dari argument constructor disimpan sebagai state object Pesanan.
menu.isTersedia();	Pesanan meminta informasi kepada MenuItem melalui method call.
private static int nextNumber	Counter dimiliki class dan dibagi seluruh instance.

Tahap 4 - Buat dan Jalankan Object yang Berkolaborasi

Main - menghubungkan object

```
Pesanan p1 = new Pesanan(m1, nasi, 2);
Pesanan p2 = new Pesanan(m2, kopi, 1);

System.out.println("P1 dapat diproses: " + p1.dapatDiproses());
System.out.println("Total P1: " + p1.hitungTotal());

System.out.println("P2 dapat diproses: " + p2.dapatDiproses());
```

Observasi	Penjelasan
Mengapa p1 dan p2 dapat memiliki pemesan berbeda?	
Mengapa p2 dapat ditolak setelah kopi ditandai habis?	
Object mana yang mengetahui harga? Mengapa bukan Pesanan yang menyimpan copy harga?	
Di baris mana message antar-object terjadi?	

Tahap 5 - Uji Static secara Sengaja

1. Tambahkan getter getNomor() pada Pesanan.
2. Cetak nomor p1 dan p2.
3. Buat p3 dan cetak nomornya.
4. Prediksi hasil jika kata static pada nextNumber dihapus.
5. Hapus static, jalankan, dan bandingkan hasil.
6. Kembalikan static setelah eksperimen selesai.

Kesimpulan yang dicari

nomor adalah state per-object; nextNumber adalah state shared untuk seluruh class Pesanan. Pemilihan static harus mengikuti kepemilikan data, bukan sekadar agar mudah dipanggil.

Tahap 6 - Susun Package

1. Pindahkan Mahasiswa, MenuItem, dan Pesanan ke package id.ac.polban.pbo.kantin.model.
2. Pindahkan Main ke package id.ac.polban.pbo.kantin.app.
3. Tambahkan package declaration pada setiap file.
4. Tambahkan import pada Main.
5. Compile/run kembali dan perbaiki error satu per satu.
6. Pastikan semua skenario sebelumnya tetap berjalan.

> **Checkpoint praktikum**
>
> Sebelum masuk task mandiri, Anda harus dapat menunjuk satu baris kode untuk masing-masing istilah: class, object, state, behavior, encapsulation, constructor, static, relationship, dan package.

5. Eksperimen Teknis

Eksperimen berikut sengaja meminta Anda merusak atau mengubah kode. Tujuannya agar konsep terlihat melalui perbedaan hasil, bukan hanya dibaca sebagai definisi.

Eksperimen 1 - private vs public

1. Pastikan field tersedia pada MenuItem adalah private.
2. Dari Main, tulis menu.tersedia = false; dan compile.
3. Catat error compiler.
4. Ubah field sementara menjadi public dan compile kembali.
5. Jalankan program, lalu kembalikan field menjadi private.

Pertanyaan	Jawaban
Apa yang berubah ketika private menjadi public?	
Apakah public membuat program lebih “fleksibel”? Risiko desain apa yang muncul?	
Mengapa tandaiHabis() lebih bermakna daripada akses langsung ke field?	

Eksperimen 2 - Constructor vs Object Setengah Jadi

1. Coba panggil new Mahasiswa() tanpa argument.
2. Amati error setelah class hanya memiliki constructor berparameter.
3. Tambahkan sementara constructor kosong, lalu buat object tanpa NIM/nama.
4. Tanyakan: apakah object itu masuk akal untuk kasus kita?
5. Hapus constructor kosong setelah observasi.

> **Makna teknis**
>
> Constructor bukan sekadar method khusus yang namanya sama dengan class; constructor menentukan bagaimana object boleh mulai hidup di program.

Eksperimen 3 - static vs instance

1. Jalankan tiga object Pesanan saat nextNumber static.
2. Catat nomor setiap object.
3. Hapus static, reset nextNumber = 1 sebagai field instance, jalankan lagi.

4. Jelaskan mengapa hasil berbeda.

Eksperimen 4 - Relationship: Reference vs Duplikasi Data

Bandingkan dua desain berikut secara konseptual. Jangan implementasikan Desain B secara permanen. **Dua cara menyimpan informasi pada Pesanan**

```
// Desain A
private Mahasiswa pemesan;
private MenuItem menu;

// Desain B
private String nimPemesan;
private String namaPemesan;
private String kodeMenu;
private String namaMenu;
private int hargaMenu;
```

Pertanyaan	Jawaban
Desain mana yang menunjukkan relationship antar-object secara langsung?	
Jika nama mahasiswa berubah, desain mana yang berpotensi menyimpan copy lama?	
Apakah menyimpan object reference selalu otomatis benar? Jelaskan berdasarkan kebutuhan kasus.	

Eksperimen 5 - Package dan Compiler

1. Hapus satu import model dari Main. Compile.
2. Catat jenis error.
3. Tambahkan import kembali.
4. Ubah package declaration satu class sehingga tidak cocok dengan lokasinya. Amati error/masalah dari IDE/compiler. 5. Kembalikan struktur package ke kondisi benar.

6. Tugas Praktikum Terintegrasi

Kerjakan pada project yang sama. Kode akhir harus menunjukkan konsep-konsep modul melalui kasus, bukan hanya memenuhi output.

Task 1 - Lengkapi Model Teknis

1. Lengkapi Mahasiswa, MenuItem, dan Pesanan dengan constructor, getter yang memang diperlukan, serta behavior yang relevan.
2. Jangan membuat setter untuk semua field secara otomatis. Setiap setter/behavior harus memiliki alasan. 3. Tambahkan method ringkasan() atau method sejenis pada Pesanan untuk menghasilkan informasi yang mudah diperiksa.
4. Pastikan field domain private.

Task 2 - Buktikan Class dan Object

1. Buat minimal dua Mahasiswa dan dua MenuItem.
2. Buat minimal tiga Pesanan.
3. Pastikan masing-masing object dapat menunjukkan state yang berbeda.
4. Tulis komentar singkat pada worksheet: mana yang merupakan class dan mana yang merupakan

object/instance. **Task 3 - Buktikan Encapsulation dan Constructor**

1. Sertakan satu screenshot/error compiler ketika mencoba mengakses field private dari Main. 2. Sertakan satu bukti bahwa object tidak dapat dibuat dengan constructor yang tidak tersedia.

3. Jelaskan mengapa constructor yang Anda pilih sesuai untuk state awal object.

Task 4 - Buktikan Static

1. Gunakan static counter untuk nomor pesanan.
2. Cetak nomor tiga Pesanan.
3. Jelaskan mengapa counter static tetapi nomor tidak static.
4. Tambahkan static method getJumlahPesananDibuat() yang mengembalikan jumlah object Pesanan yang sudah diberi nomor.

Contoh static method yang membaca static field

```
public static int getJumlahPesananDibuat() {
 return nextNumber - 1;
}
```

Task 5 - Buktikan Relationship

1. Pesanan harus menyimpan reference ke Mahasiswa dan MenuItem.
2. dapatDiproses() menggunakan state MenuItem melalui method, bukan dengan mengakses field langsung. 3. hitungTotal() menggunakan harga MenuItem melalui method.
4. Gambarkan hubungan Mahasiswa - Pesanan - MenuItem dan hubungkan dengan field reference pada source code. **Task 6 - Package Final**

Package minimum

```
id.ac.polban.pbo.kantin.model
 Mahasiswa
 MenuItem
 Pesanan

id.ac.polban.pbo.kantin.app
 Main
```

1. Pastikan semua file memiliki package declaration yang benar.
2. Main menggunakan import yang diperlukan.
3. Program dapat dijalankan dari IDE/terminal sesuai setup kelas.

6.1 Skenario Uji Wajib

ID	Skenario	Expected Result
T-01	Buat dua Mahasiswa.	Keduanya berasal dari class sama tetapi memiliki state berbeda.
T-02	Akses field private dari Main.	Compiler menolak akses.
T-03	Tandai salah satu MenuItem habis.	State menu lain tidak berubah.
T-04	Buat tiga Pesanan.	Nomor meningkat 1, 2, 3 sesuai awal program.
T-05	Pesanan dengan menu tersedia dan jumlah > 0.	dapatDiproses() bernilai true.

T-06	Pesanan dengan menu habis.	dapatDiproses() bernilai false.
T-07	Pesanan jumlah 0.	dapatDiproses() bernilai false.
T-08	Package + import final.	Program compile/run tanpa error package/import.

6.2 Bukti Wajib

• Source code final dengan struktur package.
• Screenshot/output T-01, T-04, T-05, T-06.

Tabel hasil eksperimen static sebelum/sesudah.

• Satu bukti error compiler untuk encapsulation. •

• Diagram relationship sederhana yang dapat ditautkan ke field reference pada kode.
• Jawaban refleksi konsep dengan bahasa sendiri.

7. Independent Challenge

7.1 Challenge A - Relationship uses-a dengan Kasir

Tambahkan class Kasir pada package app atau service sederhana. Kasir tidak perlu menyimpan Pesanan sebagai field. Ia hanya menerima Pesanan saat method dipanggil.

Contoh uses-a: object hanya digunakan saat operasi

```
public class Kasir {
 public void proses(Pesanan pesanan) {
 if (pesanan.dapatDiproses()) {
 System.out.println("Pesanan diproses");
 } else {
 System.out.println("Pesanan ditolak");
 }
 }
}
```

Bandingkan dengan relationship has-a pada Pesanan → Mahasiswa/MenuItem. Jelaskan perbedaan keduanya dengan merujuk langsung pada struktur field dan parameter method.

7.2 Challenge B - Pertanyaan Menuju Inheritance

Jika nanti terdapat MenuMakanan dan MenuMinuman, apakah keduanya is-a MenuItem? Tuliskan alasan konseptual. Jangan implementasikan inheritance pada modul ini.

8. Checkpoint, Penilaian, dan Refleksi

8.1 Definition of Done

1. Mahasiswa dapat menunjukkan class dan minimal dua object dari class yang sama.
2. Field domain utama menggunakan private.
3. Perubahan state penting dilakukan melalui method yang dapat dijelaskan.
4. Constructor digunakan untuk state awal yang dibutuhkan.
5. Perbedaan static dan instance dapat dibuktikan melalui output.
6. Pesanan menyimpan object reference untuk merepresentasikan relationship.

7. Program menggunakan package model dan app.
8. Semua skenario T-01 s.d. T-08 memiliki hasil yang dapat diperiksa.
9. Mahasiswa dapat menunjuk baris kode yang merepresentasikan setiap konsep utama.

8.2 Self Assessment

Skala: 1 belum paham; 2 paham dengan banyak bantuan; 3 dapat mengikuti panduan; 4 mandiri; 5 dapat menjelaskan kepada orang lain.

Pernyataan	Skor 1-5
Saya dapat membedakan class dan object pada source code.	
Saya dapat menjelaskan state dan behavior dari object yang saya buat.	
Saya memahami mengapa field dibuat private dan kapan sebuah method digunakan untuk mengubah state.	

Pernyataan	Skor 1-5
Saya dapat menjelaskan fungsi constructor pada kasus ini.	
Saya dapat membedakan field instance dan static melalui hasil program.	
Saya dapat menunjukkan relationship antar-class melalui object reference.	
Saya dapat menjelaskan fungsi package dan import.	
Saya dapat mengubah bagian kecil program tanpa AI dan menjelaskan dampaknya.	

8.3 Lesson Learnt

1. Baris kode mana yang paling membantu Anda memahami perbedaan class dan object?
2. Apa perbedaan antara “field private” dan “object benar-benar menjaga state-nya”?
3. Apa yang terjadi ketika constructor kosong diizinkan? Apakah object masih masuk akal pada kasus ini? 4. Mengapa nextNumber cocok static tetapi nomor tidak?
5. Bagaimana relationship antar-class terlihat secara nyata di source code?
6. Apa fungsi package selain membuat folder terlihat rapi?
7. Satu konsep apa yang masih paling membingungkan setelah praktikum?

9. Referensi

1. Oracle Java Tutorials / Java Language Documentation: Classes and Objects, Constructors, Access Control, Class Members, Packages.
2. Catatan kuliah Pemrograman Berorientasi Objek Pertemuan 1-3.
3. Bloch, Joshua. Effective Java. Addison-Wesley. Digunakan sebagai referensi umum praktik desain object; tidak seluruh topik menjadi materi wajib modul.